Travelling Salesman Problem

Introduction

In this project, I work with a 50-city Euclidian TSP instance which I use to compare a single-solution Tabu Search, and a population based Genetic Algorithm.
This report focuses on explaining design choices, interpreting the results and reflecting on what I learnt during implementation.

1. Problem Definition

The TSP problem is defined as follows: given a set of cities and distances between them, find the shortest possible tour that visits each city exactly once and returns to the starting city.
each city is represented by a pair of coordinates (x, y) provided in the ‘cities.csv dataset’.
These coordinates help construct a Euclidian distance between the i and j city and the objective function becomes the total length of the closed tour computed using this matrix.
A good solution would be to arrange the indices of all the cities and the implementation checks to ensure every solution satisfies the constrain of visiting each city just once.
The main problem contains 50 cities but I used smaller subsets of 10, 20, 30 and 40 cities to check for scalability. The number of possible tours grows factorially with the number of cities making exhaustive search not ideal so heuristic and metaheuristic approaches are required to produce near-optimal solutions. (Glover and Kochenberger, 2003)

2. Algorithms Implemented

(a) Tabu Search (Single-Solution Algorithm)

This is a metaheuristic algorithm that guides local search by using short-term memory to prevent cycling and encourage exploration beyond the local minima. It is a widely used optimisation literature as effective for permutation-based problems such as The Travelling Salesman Problem (Glover, 1989)

In my implementation, the neighbourhood uses a swap operator to change two positions in the tour. These swap moves are computationally cheap and ensure all solutions are valid permutations. I also implemented adaptive tenure to improve the robustness of the algorithm whereby, if the algorithm shows no improvement, the tabu tenure increases to promote diversification. (Glover and Laguna). When improvement occurs frequently, it decreases to focus more on intensification.

I implemented a 2-opt neighbourhood to address inefficient edge crossings of the tour, this allowed tabu search to escape local minima
The (Figure 1) shows the swap variant converging rapidly and stagnates early but the 2-opt variant improves and achieves a substantially lower final tour cost.

(b) Genetic Algorithm (Population-based evolutionary algorithm)

They are inspired by natural evolution and maintain a population of candidate solutions that evolve through, selection, crossover and mutation (Holland, 1975; Goldberg, 1989). They explore many regions of the solution space in parallel making them more effective to escape local optima in highly multimodal scenarios.
In this project, tours are represented as permutation chromosomes and viability is preserved by using Partially Matched Crossover which exchanges structured segments between parents while repairing duplicates. Tournament

selection chooses fitter parents with higher probability, balancing exploration and exploitation. A swap mutation operator introduces controlled randomness by exchanging two random positions with a given probability. Elitism ensures that the best solution of each generation is always retained.
Genetic Algorithm demonstrates smoother but slower convergence curves (Figure 2) and outperforms Tabu Search on the final tour length.
The scalability shows performance reduction in a better manner compared to Tabu Search as city count increases.

3. Experimental Setup

All experiments were carried out in Python using a Jupyter Notebook, with the code organised into clear sections so each part of the project could be tested and reused easily. The dataset used contains 50 cities with their (x, y)coordinates stored in cities.csv. From this full set, I also created smaller subsets of **10, 20, 30, and 40 cities** so I could observe how each algorithm scaled as the problem grew.

A Euclidean distance matrix was generated once at the start and reused by all algorithms. This ensured fairness and kept the results consistent. Random seeds were set where appropriate to reduce unnecessary variation.

For **Tabu Search**, I used a swap neighbourhood, a 2-opt swap, a 200-iteration limit, and an **adaptive tabu tenure** that increases when the search stagnates and decreases after improvements. This allowed the algorithm to switch naturally between exploring new areas and refining promising tours.

For the **Genetic Algorithm**, I used a population size of 60, tournament selection, PMX crossover, swap mutation at 0.1 probability, and elitism. These settings were chosen because they reliably produced improvements without making the runtime too long.

To evaluate performance, I compared solution quality, runtime, and convergence behaviour across all instance sizes. Finally, because these algorithms involve randomness, I ran each one **20 times** on the 50-city instance and applied a **paired t-test** to check whether any performance difference between Tabu Search and the GA was statistically meaningful. This follows guidance from Field (2013) and the original work of Student (1908).

4. Results and Analysis

This section is a summary of the two algorithms using outputs, tables and plots I the notebook.

4.1. 50-city instance performance

As shown in Table 1, GA finds a shorter tour compared to Tabu Search though the difference is not large. The convergence curves in Figure 1 show tabu search improving quickly in the first few iterations and then plateaus and Figure 2 shows GA progressing more gradually across generations and ends up improving even after Tabu Search stabilises. This happens because tabu search relies on a single trajectory, settling into a local optimum while GA benefits from population diversity and crossover reaching better global solutions at the cost of longer runtime.

4.2. Effect of Neighbourhood choice in Tabu Search

The swap-based Tabu search variant shows rapid early improvement but plateaus, indicating limited ability to escape local minima. The 2-opt variant however continues improving beyond the initial convergence phase and achieves a lower final tour cost.

4.3. Scalability across 10-50 cities

For smaller instances (10-20 cities), both algorithms perform similarly and reach near optimal tours easily. As the problem size grows, Tabu Search begins to show stagnation signs with its tour cost increasing more sharply than GA.
GA shows more stable performance across different sizes, and its runtime also increases with population size and general count. The results show tpopulation-

based algorithms are stronger when search spaces become highly multimodal such as with the TSP project. (Goldberg, 1989)

4.4. Statistical Comparison using 20-run paired t-test

A 20-run paired t-test is performed using the best tour cost from each run since single runs are because of randomisation from both algorithms. The results on Figure 3 and the t-test output check whether the average difference is statistically significant.

I checked if the p-value was below 0.05 which should suggest that one algorithm consistently produced better results than the other and, in this case, GA achieved lower average costs with less variability and the t-test confirmed that it was better than Tabu Search. (Field, 2013)

4.5. Findings summary

GA demonstrated higher solution quality and more reliable performance, and the statistical analysis strengthened the conclusions by showing the GA improvements were not random. (Glover, 1989; Goldberg, 1989)

5. Critical Discussion

A critical behavioural difference is observed; Tabu Search is effective at quickly improving a single solution by exploring its local structure as shown by the steep early decline in its convergence curve (Figure 1). It relies heavily on adaptive tenure to escape and even then; it sometimes becomes trapped which explains the weaker performance on larger instances.

GA benefits from maintaining a population of diverse solutions, enabling exploration of different areas of the search landscape simultaneously. This reduces premature convergence and allows it to continue improving later into the search process. The PMX crossover helps preserve useful subsequence of tours while mutation gains

variety. A key trade-off is **runtime versus robustness**. Tabu Search is consistently faster due to its lightweight neighbourhood evaluations, making it attractive when quick solutions are needed. Although 2-opt neighbourhoods are theoretically more expensive, the observed runtime difference on the 50-city instance was small and can be attributed to the modest problem size and similar neighbourhood counts.

The GA, although slower, adapts more naturally to increasing problem complexity through its population dynamics.

6. Reflections on learning and process

Working on the TSP project enabled me to get a clear understanding of both algorithms that I used. The graphs especially help me visualise how they react to the problem structure, neighbourhood choice and parameter settings. Using Simple hill climbing as a baseline helped me understand how easily a single solution can get trapped and how adaptive tenure helps solve that.
Running the t-test helped confirm the importance of applying statistical evidence rather than judging performance based on a few runs especially when it involves randomness.

Overall, it was fun to see the theoretical concepts taught in class translate into real behaviour when implemented on an optimization problem.

7. Ethical, Legal & Academic Integrity considerations

I ensured that the project complied with ethical and academic integrity expectations. I used generative AI tools, specifically ChatGPT to help me with coding some sections and do a lot of the debugging. I also modified some of the code provided to us in our classwork by the module lecturer. Any ideas or concepts from literature was properly referenced.

8. Conclusion

I successfully implemented both algorithms and evaluated them using consistent metrics for solution quality, runtime, convergence behaviour, scalability and statistical significance. Tabu Search is well suited for instances where fast approximate solutions are required. Genetic Algorithm is better suited for instances where broader explorations are required.

This project showed me the importance of careful experimental design, statistical evaluation and reflective analysis when choosing optimisation algorithms.

8.1. Recommendations for improvement

Combining genetic Algorithm with Tabu Search could help with the achieving the global exploration strengths and quick local refinement of single-solution heuristics.

8.2. Potential directions for future research

This could be exploring adaptive parameter control, allowing parameters such as tabu tenure or population size to evolve dynamically during the search process rather than remaining fixed.

9. References

Field, A. (2013) Discovering statistics using IBM SPSS statistics. 4th edn. London:Sage

Glover, F. (1989) 'Tabu Search – Part I', ORSA Journal on computing, 1(3), pp.190-206

Glover, F. and Kochenberger, G.A. (2003) handbook of metaheuristics. Boston:Springer.

Glover, F. and Laguna, M. (1997) Tabu search. Boston:Springer.

Goldberg, D.E. (1989) Genetic Algorithms in Search, Optimisation and Machine Learning. Reading, MA: Addison-Wesley.

Holland, J.H. (1975) Adaptation In Natural and Artificial Systems. Ann Arbor: University of Michigan Press.

Lawler, E.L., Lenstra, J.K., Rinnoy Kan, A.H.G and Shmoys, D.B. (1985) The Travelling Salesman Problem: A guided tour of combinatorial optimisation. Chichester:Wiley.

Student (1908) ‘The probable error of a mean’, Biometrika, 6(1), pp. 1-25